for unit-testing libraries that still require the "test" prefix on unit-test method names, as illustrated in the following Python examples:

```
class WhenUpdatingStatusPoints(unittest.TestCase):

    def test_that_status_points_can_be_added(self):
        ...

    def test_that_status_is_updated_when_enough_points_are_earned(self):
        ...
```

In all of these cases, you're describing what a class should do, rather than testing individual methods. This makes the tests read more like technical documentation for the class that describes and illustrates the intended usage of the class, rather than just listing method names prefixed with the word "test." It also makes the test cases more robust: method names, and how you use them, are much more likely to change during refactoring than the expected behavior of the class.

USE TEST CLASS NAMES THAT PROVIDE CONTEXT

In a similar vein, BDD practitioners like to name the test classes in a more meaningful way. The name of a BDD-style unit-test class isn't related to the class being tested (which, when you create the test class, may not even exist). Rather, it gives the context of the behavior being described.

A useful trick for doing this is to prefix the test class name with the word "When." This approach, combined with the more expressive test names discussed earlier, makes the tests read much more like technical specifications:

```
public class WhenUpdatingMemberStatusPoints {

  @Test
  public void should_be_able_to_add_status_points_earned_in_a_flight(){
    ...
  }

  @Test
  public void should_update_status_when_enough_status_points_are_earned(){
    ...
  }
  ...
}
```

The test class name describes the feature or context.

The test names describe the detailed requirements.

These techniques let you obtain many of the benefits of BDD unit testing without needing to change your unit-testing tools. But BDD unit-testing tools that are more focused, such as RSpec for Ruby, NSpec for .NET, and Spock for Java and Groovy, make things a little easier. They make it simpler and more natural to write unit tests in the form of clear, readable, low-level executable specifications. In the next section, we'll look at how they can help.

10.4.2 Writing specifications, not tests: the RSpec family

Inspired by the concepts of BDD, in the mid-2000s a new generation of low-level BDD tools emerged that put a greater distance between "executable specifications" and